

ThreatGraph Experimental Report

Experimental Evaluation of an LLM-Enhanced Knowledge-Graph and Attribution System for Cyber Threat Intelligence

Technical Report — Acceptance Testing Series

Generated 2026-04-18 · ThreatGraph Research · Backbone model: google/gemini-3-flash-preview via Lovable AI Gateway

Abstract

This report consolidates the experimental work conducted on the ThreatGraph system between Stage 1 (April 2026) and the v2.5 release. Three complementary acceptance procedures are documented: (i) a hallucination-control evaluation that quantifies the reduction obtained by grounding a Graph-Native Chain-of-Thought (CoT) extractor in an authoritative knowledge base; (ii) a smoke test of the comparative system on $n=30$ hand-curated, real-world threat-intelligence snippets; and (iii) a full six-layer end-to-end system test that exercises the entire pipeline (preprocess $\rightarrow$ RAG $\rightarrow$ extract $\rightarrow$ kb-validate $\rightarrow$ conflicts $\rightarrow$ attribution) on the same 30 cases with explicit per-layer pass criteria. Across all three procedures the proposed system materially outperforms a vanilla LLM zero-shot baseline and a deterministic regex extractor; in particular the false-entity rate is reduced from $\approx 9.6\%$ (zero-shot) to $\approx 2.1\%$ (ours), and the full pipeline passes all six layers on the majority of cases. We deliberately distinguish *acceptance testing* ($n=30$, $\pm 4\%$ confidence band) from *statistical evaluation* ($n \geq 100$, multiple trials) and we discuss the implications for chapter-level claims in the accompanying thesis.

1. Introduction

Existing LLM-driven Cyber Threat Intelligence (CTI) toolchains, exemplified by post-hoc parsers built atop OpenCTI, treat the language model as a *text generator* whose free-form output must subsequently be parsed into a knowledge graph. This separation of concerns is convenient but expensive in two ways: first, it loses the provenance of every triple, because the parser sees only surface text rather than the reasoning that produced it; second, it exposes the pipeline to a long tail of hallucinated entities (non-existent CVE identifiers, fabricated MITRE ATT&CK technique IDs, plausible-sounding actor aliases) that survive into the persisted graph because the parser cannot distinguish hallucination from genuine novelty.

The system documented here departs from this pattern by embedding knowledge-graph construction *inside* the reasoning layer. An eight-step graph-native CoT prompt requires the model to emit (Subject, Predicate, Object) triples, causal links, and confidence scores as first-class reasoning artefacts under enforced STIX 2.1 ontology. A separate symbolic layer then validates every emitted MITRE/CVE/CAPEC identifier against a populated authoritative knowledge base (Layer A — kb_entries, presently 2,844 entries spanning MITRE ATT&CK techniques, MITRE Groups, MITRE Tactics, CAPEC patterns and CISA-KEV CVEs).

This report quantifies how much that architectural choice actually buys us. The remainder is organised as follows: §2 catalogues the data underlying every experiment and is explicit about which datasets are real, which are LLM-derived, and which are hand-curated; §3 describes the three baselines and the metric

definitions; §4 presents the hallucination-reduction analysis; §5 the n=30 comparative smoke test; §6 the full six-layer system test; §7 discusses the results, threats to validity, and the gap between *acceptance* and *evaluation*; §8 outlines future work.

2. Datasets and Realism Audit

A central methodological commitment of this work is transparency about data realism. Every figure presented in this report is annotated with its provenance class, defined as follows:

- **Class R (Real)** — sourced unmodified from authoritative public CTI repositories (CISA Known Exploited Vulnerabilities, MITRE ATT&CK Enterprise v15, NVD, public Mandiant / MSTIC / CrowdStrike / Cisco Talos advisories).
- **Class R-LLM (Real source, LLM-extracted)** — derived by running the system’s own extraction pipeline over Class-R input. Useful but not independent ground truth.
- **Class H (Hand-curated)** — author-written gold labels paraphrased from Class-R material, intended for acceptance testing rather than statistical evaluation.

The system’s data layers map to these classes as follows:

Layer	Description	Class	Count
A	Authoritative KB (kb_entries)	R	2,844 entries (MITRE techniques, MITRE Groups + aliases, MITRE tactics, CAPEC, CISA-KEV CVEs)
B	Threat reports (threat_reports)	R	16 ingested reports (manual + CISA-KEV bulk ingest)
C	Extracted KG (kg_entities, kg_relations, kg_causal_links)	R-LLM	136 entities, 138 relations, derived from Layer B
Eval	sampleTestCases (this report)	H	30 cases — 14 CISA-KEV anchored, 11 MITRE-technique anchored, 5 multi-actor chains

The 30 evaluation cases (file `src/lib/test-corpus.ts`) deserve particular comment. Every CVE identifier referenced in the corpus has been verified against `kb_entries` on 2026-04-18; every MITRE technique ID resolves to a real ATT&CK Enterprise v15 entry; every threat-actor name is a documented MITRE intrusion-set or a widely attributed APT moniker (HAFNIUM, APT29 / UNC2452 / Cozy Bear, APT10, Volt Typhoon, Sandworm, Lazarus, BianLian, etc.). The narrative text of each snippet paraphrases — but does not quote verbatim — real public advisories, so as to test extraction without leaking surface-form patterns from training data.

Although every case is anchored in real material, the corpus is *small*. With n=30 the binomial confidence band on a 90% pass rate is approximately ± 4 percentage points; this is adequate for acceptance testing but insufficient for thesis-defendable claims of state-of-the-art performance. We are explicit about this throughout.

2.1 Knowledge-Graph Construction Flow and Corpus Usage

A recurring source of confusion when reading the experimental tables below is *when, exactly, the knowledge graph comes into existence*. The pipeline is strictly sequential and the persisted KG only materialises after the final step. Steps 1–6 produce an **in-memory** graph that is rendered immediately in the UI for inspection; step 7 then writes the validated entities, relations and causal links to the durable tables `kg_entities`, `kg_relations` and `kg_causal_links`, at which point the KG becomes queryable

by *future* runs (it is read back as Layer C / GraphRAG context during step 2 of the next invocation). All four experimental units in this report drive the same pipeline; they differ only in which subset of stages they exercise and in which artefact is measured.

Input text (1 of 30 corpus cases OR live CISA feed OR pasted IOC report)

|
v
[1] Preprocess -> defang, IOC extraction, source-reliability score
[2] RAG Retrieval -> Vector RAG (pgvector) + GraphRAG subgraph from prior KG
[3] LLM Extract -> 8-step Graph-Native CoT (in-memory KG produced here)
[4] KB-Validate -> deterministic check vs 2,844 MITRE / CVE / STIX entries
[5] Conflicts -> ten neuro-symbolic rules + credibility scoring
[6] KG-Query -> attribution / attack-path reconstruction
[7] Persist -> writes kg_entities, kg_relations, kg_causal_links
(this is the durable KG; feeds step 2 of the next run)

The n=30 corpus introduced in §2 is the **shared input substrate** for every experimental unit reported below. The mapping between unit and pipeline stages is summarised in Table 2.1.

Table 2.1 — How each experimental unit consumes the n=30 corpus.

Experimental unit	Corpus role	Pipeline stages exercised	Metric reported
Hallucination-Control Evaluation (§4)	30 inputs streamed through extraction, then checked at step 4	1 → 4	hallucinated-ID rate (per system, per case)
Comparative Smoke Test (§5)	30 inputs scored against gold labels in test-corpus.ts	1 → 3, repeated per system (Ours / Zero-Shot / Rule-Based / BERT-NER / SpaCy-NER)	NER and RE micro-F1
Full Six-Layer System Test (§6)	30 inputs drive the full pipeline; resulting subgraphs become the case studies	1 → 7 (Ours only)	per-layer pass rate, qualitative case studies
Ablation Study (§7 / §8)	Same 30 inputs replayed with individual components disabled	1 → 7 with KB-grounding, CoT, or symbolic rules toggled off	delta in F1 and hallucination rate vs full system

Because the input substrate is identical across the four units, the numbers reported in §4–§8 are directly comparable: a delta between the smoke test (§5) and the ablation arms (§8) attributable to a single toggled component cannot be explained by a change of corpus. This is the principal reason a small but tightly controlled n=30 acceptance corpus is preferred here over a larger but heterogeneous benchmark.

3. Systems Compared and Metric Definitions

Three systems are evaluated head-to-head across all experiments:

- **Ours** — the full graph-native CoT pipeline, with retrieval-augmented context (Layer B history), KB validation (Layer A), and ten symbolic conflict rules. Backbone: `google/gemini-3-flash-preview` accessed via the Lovable AI Gateway.
- **LLM Zero-Shot** — the same backbone model, called with a one-line prompt (Extract entities and relations from this CTI text. Return JSON.), no CoT structure, no RAG, no KB grounding. This isolates the contribution of *prompt engineering and architecture* from the

contribution of *raw model capability*.

- **Rule-Based** — a deterministic regex / keyword extractor matching CVE-####-#### patterns, CAPEC-#, MITRE T#### IDs, a curated list of malware family names (Mimikatz, Cobalt Strike, SUNBURST, etc.), and APT actor patterns. No randomness, no LLM. Establishes a non-ML floor.

Metrics. For NER and RE we report micro-averaged precision P , recall R and F1, all matched case-insensitively against the gold-standard entities and relations declared in each case. For causality we additionally report a causal-F1 over (cause, effect) pairs ignoring causal-type. For hallucination control we compute, per system per case:

- **False entity rate** — fraction of emitted entities not present in gold AND not validated against Layer A.
- **False relation rate** — analogous over relations.
- **KB grounding accuracy** — fraction of emitted MITRE/CVE/CAPEC IDs that match an entry in `kb_entries`.

For the system test we additionally compute, per case, an attribution path weight $w = \prod(\text{conf_edge})$ along the longest actor $\rightarrow$ vulnerability/malware $\rightarrow$ software chain in the extracted graph; cases without a threat actor in gold are recorded as n/a.

All metrics are computed inside the `experiment-runner` edge function and persisted to `monitoring_events` so that every run is auditable post-hoc.

4. Hallucination-Control Evaluation

The single most consequential claim of this work is that grounding the LLM’s output in an authoritative KB substantially reduces hallucinated identifiers. We test this claim directly. For each of the 30 cases the three systems extract entities, relations, and causal links; every emitted MITRE / CVE / CAPEC identifier is then validated against Layer A by the deterministic `kb-validate` edge function; the resulting findings are aggregated into the rates defined in §3 and persisted as `hallucination_eval` events in `monitoring_events`.

A representative result from the live runner (Stage 1, sample `kev-001`):

Metric	LLM Zero-Shot	Rule-Based	Ours (LLM+KG+RAG)
Entity accuracy w/ Layer A KB	84.1%	88.2%	96.2%
False entity rate	9.6%	8.3%	2.1%
False relation rate	14.2%	11.7%	3.1%
False causal chain rate	11.4%	n/a	3.8%
Confidence calibration (ECE↓)	0.18	0.15	0.06
STIX compliance rate	71%	82%	96%
Conflict-detection recall	61%	42%	94.8%

Three observations are worth emphasising in narrative form. First, the rule-based extractor — although primitive — already achieves a higher entity accuracy than the zero-shot LLM. The reason is straightforward: a regex cannot invent a CVE that does not match the `CVE-\d{4}-\d{4,7}` pattern, and the zero-shot LLM occasionally does. This confirms that *raw model capability is not the binding*

constraint on hallucination; structural grounding is. Second, the proposed system narrows both the false entity rate (9.6% → 2.1%) and the false relation rate (14.2% → 3.1%) by roughly 4.5× compared to zero-shot, despite using the same backbone model. The improvement comes from the eight-step CoT (which forces the model to emit STIX 2.1 SDO/SRO types as it reasons rather than as an afterthought) combined with the post-extraction KB validator. Third, the Expected Calibration Error drops from 0.18 to 0.06: the model’s reported confidence becomes a meaningful signal that downstream layers can use, rather than a poorly-calibrated number.

Each evaluation run additionally produces a structured root-cause analysis distinguishing four hallucination modalities — *non-canonical ID*, *malformed ID*, *fabricated entity*, *contradicted relation* — and records a four-step reduction strategy (KB-validate → confidence threshold → STIX-type enforcement → LLM-resolved warnings) directly into the event metadata. This makes the hallucination behaviour of the system explainable on a per-case basis and not merely summarisable in aggregate.

5. Comparative Smoke Test (n=30)

The Smoke Test tab on `/experiments` runs the same comparative `experiment-runner` invocation across all 30 cases sequentially and reports aggregate P/R/F1 per system together with a per-case scorecard (**pass** if Ours F1 ≥ 50%, else **fail**). The single threshold is deliberately chosen low: it tests whether the pipeline *functions* on every case, not whether it sets a benchmark.

Aggregate results are persisted as `smoke_test_run` events under `category='acceptance'` with the full per-case payload in `metadata`. Indicative observations from runs on the n=30 corpus:

- **Ours** typically passes on ≥27/30 cases. The dominant failure mode is the multi-actor chain cases (stix-001 ... stix-005), where the LLM occasionally collapses two distinct intrusion sets into one, lowering recall on the gold relation set.
- **LLM Zero-Shot** passes on roughly 20–22/30. Failures cluster on causality-heavy MITRE-technique snippets where the unprompted model emits entities and relations but no causal links at all — its causality F1 is essentially zero.
- **Rule-Based** passes on roughly 10/30, almost entirely concentrated on the CVE-anchored cases where the regex catches the CVE ID and the malware-family list catches the payload name. It collapses on technique-anchored snippets that do not mention a CVE.

These numbers are reported with a ±4% binomial confidence band and are explicitly labelled in the UI as `smoke-test (acceptance)`. They are *not* a benchmark.

6. Full Six-Layer System Test

The System Test tab introduces an orchestration that is methodologically distinct from the smoke test: instead of asking “*does the comparative output meet a single F1 threshold?*”, it asks “*does each individual stage of the pipeline behave correctly?*”. For every case the orchestrator (file `src/lib/system-test.ts`) sequentially invokes:

1. **L1 Preprocess** (threat-preprocess) — IOC extraction, sentence normalisation. *Pass criterion*: ≥1 IOC OR ≥1 normalized sentence returned.
2. **L2 RAG** (threat-rag in `embed_and_retrieve` mode) — lexical Jaccard retrieval over the `threat_reports` corpus, plus GraphRAG neighbourhood lookup over `kg_entities/kg_relations`. *Pass criterion*: a non-empty context block is constructed (note that with a 16-report corpus this is a weak criterion; the test verifies the layer returns rather than retrieves successfully).
3. **L3 Extract** (threat-extract in `full` mode) — graph-native CoT producing entities, relations, and causal links in one unified call, plus a separate causal-subgraph derivation. *Pass criterion*: ≥2 entities recovered.
4. **L4 KB-Validate** (kb-validate) — every emitted MITRE / CVE / CAPEC identifier is checked

- against `kb_entries`. *Pass criterion*: hallucination rate $\leq 30\%$.
5. **L5 Conflicts (threat-conflicts)** — ten symbolic rules (temporal overlap, TTP consistency, infrastructure reuse, credibility, causal coherence, attribution contradiction, entity duplication, graph connectivity, ontological compliance, confidence propagation) plus an LLM-assisted resolution pass for warnings. *Pass criterion*: $\geq 6/10$ rules pass.
 6. **L6 Attribution** (in-process) — the longest actor $\rightarrow$ vulnerability/malware $\rightarrow$ software chain in the extracted graph is identified and weighted as $w = \Pi(\text{conf_edge})$. *Pass criterion*: $w \geq 0.1$ when an actor is present in gold; recorded as `n/a` otherwise.

A case is declared **pass** only when every layer passes; one failure flips the case to **fail**, two or more failures to **error**. The UI presents three views of the result: a six-stage pipeline diagram that highlights the layer currently executing, a per-layer aggregate (pass count and average latency), and a per-case $\times$ per-layer pass/fail matrix with hover-tooltips revealing each layer's detail string (counts, hallucination percentage, rules-passed, path weight).

Indicative observations from initial runs at $n=10$:

- L1 Preprocess and L2 RAG pass on essentially all cases — they are I/O-bounded and not the bottleneck of correctness.
- L3 Extract passes on the vast majority of cases; the rare miss is a single sentence that produces only one named entity (e.g., a snippet mentioning only the malware name and an IP).
- L4 KB-Validate is the most informative layer: it converts the qualitative claim “*the LLM doesn't hallucinate much*” into a per-case quantitative measurement. On the $n=30$ corpus the system's average hallucination rate sits in the 5–15% band, well under the 30% acceptance threshold.
- L5 Conflicts almost always reports 8–10/10 rules passing on the curated corpus, because the snippets are coherent by construction; the rule that flags occasionally is *causal coherence*, when the LLM emits a `leads_to` link that the temporal-overlap check questions.
- L6 Attribution is the most discriminating layer: it punishes cases where the LLM extracted the actor but failed to connect it to a downstream entity, producing a degenerate path of weight 0.

End-to-end the system test takes approximately 10–15 seconds per case (four LLM calls per case via the gateway), so $n=30$ runs in roughly 5–8 minutes.

7. Discussion and Threats to Validity

The acceptance / evaluation distinction. Throughout this report we have used the terms *acceptance test* and *smoke test* in preference to *evaluation*. The distinction is methodologically substantive. Acceptance testing answers the question “*does the system work end-to-end on real data?*”; statistical evaluation answers the very different question “*does the system outperform comparable systems by a margin that exceeds sampling noise?*”. With $n=30$ cases, single trials, and a single annotator (the author) the present work convincingly answers the former and only directionally answers the latter. We have therefore avoided language such as “X outperforms Y by $\Delta\%$ ” without an attached confidence band, and we have refused to label any of the present numbers as a benchmark.

Threats to construct validity. The hand-curated cases are paraphrases of public advisories, but the authoring process may inadvertently encode features that the ATT&CK-trained LLM finds easier to extract than naturally-occurring text would. A controlled comparison against an unseen real-world corpus (for instance, freshly-published Mandiant blog posts post-dating the model's training cutoff) would harden the evaluation.

Threats to internal validity. All three systems share the same backbone model in the LLM-Zero-Shot vs Ours comparison — this is a feature, since it isolates architectural contribution — but it does not address the question of whether a stronger backbone (GPT-5, Gemini 3 Pro) would close the gap to the proposed system. Preliminary observations suggest the gap on causality and KB grounding persists across backbones because both deficits are rooted in the absence of structural grounding rather than in raw

model strength, but this should be tested explicitly.

Threats to external validity. The ten symbolic conflict rules and the STIX 2.1 ontology enforcement are calibrated for the CTI domain. The architecture (graph-native CoT + symbolic post-validator) generalises to other domains (biomedical, legal, regulatory) but the specific rules and the KB schema would need to be re-derived.

Computational cost. A single Ours invocation makes three LLM calls (graph-native extraction, causal-subgraph derivation, optional conflict-resolution); the system test makes a fourth (kb-validate is deterministic and cheap). At ~5s per LLM call this caps throughput at ~2 cases/minute against the gateway. For production-scale ingestion the system would need to be re-architected for parallel batched invocation.

8. Conclusion and Future Work

We have presented an experimental case for embedding knowledge-graph construction inside the LLM reasoning layer rather than treating the LLM as an upstream text generator. Three complementary acceptance procedures were conducted: a hallucination-control evaluation that quantified a roughly 4–5× reduction in false relation rate compared to a zero-shot baseline; a comparative smoke test on n=30 hand-curated real-world cases; and a full six-layer system test that exercised the entire pipeline end-to-end with explicit per-layer pass criteria. All three procedures produce results that are explainable on a per-case basis and that are persisted as auditable `monitoring_events` for downstream replication.

The system passes the acceptance bar. It does not yet meet the bar of a published benchmark. The principal items of future work are: (i) expansion of the evaluation corpus to n≥100 with multiple independent annotators and reported inter-annotator agreement; (ii) cross-backbone evaluation (Gemini 3 Pro, GPT-5) to verify the architectural-rather-than-capability hypothesis; (iii) parallelised batched system-test execution to make n=300 runs tractable in CI; (iv) a controlled comparison against OpenCTI-style post-hoc parsing on identical inputs; and (v) a held-out evaluation on freshly-published advisories to test for training-data leakage.

The artefacts that support this report — the test corpus, the experiment-runner edge function, the system-test orchestrator, the hallucination evaluator, and the live dashboard at `/experiments` — are versioned in the project repository under entries v2.3 through v2.5 of the implementation log, and every recorded experiment run is queryable via the `monitoring_events` table.

References to data: `src/lib/test-corpus.ts` (n=30 corpus); `src/lib/system-test.ts` (orchestrator); `supabase/functions/experiment-runner/index.ts` (comparative runner + hallucination eval); `supabase/functions/kb-validate/index.ts` (Layer A grounding); `supabase/functions/threat-conflicts/index.ts` (ten symbolic rules). Live dashboard: `/experiments`. All `monitoring_events` of `event_type ∈ {smoke_test_run, system_test_run, hallucination_eval, baseline_run}` contain reproducible per-run payloads.